

Themenvorschläge für die Master-Thesis im Fernstudiengang Informatik an der Fachhochschule Trier

Thorsten Suckow-Homberg*

2026

*Fernstudiengang Informatik, Trier University of Applied Sciences, Fachbereich Informatik.
© 2026 Thorsten Suckow-Homberg, thorsten@suckow-homberg.de

Inhaltsverzeichnis

1	Einleitung	3
2	Domänenspezifische Handles zur statischen Partitionierung und konfliktar- men Ausführung einer ECS-basierten C++-Game Engine	5
2.1	Forschungsfrage	5
2.2	Methodik	5
2.3	Vorläufige Gliederung	6
3	Normalisierte ECS-Datenmodelle und domänenspezifische Handles zur sta- tischen Konfliktanalyse von Systemen in einer C++-Game-Engine	7
3.1	Forschungsfrage	7
3.2	Methodik	7
3.3	Vorläufige Gliederung	8

1 Einleitung

Die Unterstützung für Nebenläufigkeit und Parallelisierung in Game Engines ermöglicht es, unterschiedliche Gameplay- als auch Ressourcen-verwaltende Systeme verteilt laufen zu lassen, die Anzahl gleichzeitiger Berechnungen zu erhöhen und damit insgesamt zu einer Leistungssteigerung beizutragen.

Hierbei ist nicht nur die Unterstützung von Hardware und OS relevant, die Prozesse und Tasks verwaltet. Auch der Implementierung der Software kommt eine tragende Rolle zu, denn erst sie entscheidet anhand der Gruppierung und Ausführreihenfolge der Systeme, welche Abhängigkeiten der Systeme untereinander zu berücksichtigen sind, um eine deterministische Ausführungsreihenfolge zu bestimmen.

Ein wesentlicher Treiber hochperformanter Spieleengines ist dabei aber nicht nur die parallele Ausführung von Operationen, auch Lese- und Schreibdurchsatz ist relevant, bevor es überhaupt zu der Frage kommt, ob ein geteilter Speicherbereich für den Schreibzugriff gesperrt werden muss.

Hier hat sich in den vergangenen Jahren das ECS-Paradigma mit seinem datenorientierte Ansatz hervorgetan. Über das Structure-of-Arrays-Design profitieren Game Engines und Echtzeitsysteme auf Grundlage moderner CPU- und Cache-Architekturen von dichten Speicherlayouts und nah beieinanderliegender Typ- als auch Kontext-Semantisch-gleichen Daten.

Theoretischer Bezugspunkt

Die in diesem Dokument vorgestellten Forschungsfragen beschäftigen sich mit Problemstellungen, die sich aus mit dem Einsatz eines ECS-Systems ergeben. In diesem Zusammenhang haben Redmond et al. 2025 eine Arbeit vorgestellt **RCCM+25**, indem sie ein als *ECS Core* bezeichnetes formales Modell eines ECS-Systems aufsetzen. Sie zeigen, dass das Modell unter bestimmten Voraussetzungen deterministisch, unabhängig von der Ausführungsreihenfolge der Systeme, ist.

In allen vorgestellten Forschungsfragen soll diese Arbeit als theoretischer Bezugspunkt dienen; vor allem in Anlehnung an die im Modell beschriebenen Anfrage- und Scheduling-Systeme soll die Beantwortung der Forschungsfragen geschehen.

Des Weiteren bezieht sich ein Teil der Arbeit auf das Entity Relationship Model dienen. Fragestellungen zu Ausführungsreihenfolgen und konfliktarmen Scheduling wurden in

diesem Bereich bereits erforscht und liefern wertvolles theoretisches Fundament.

Implementierung für die Empirik

In diesem Zusammenhang stellen wir `helios::ecs` vor, eine ECS-Engine, die auf der im Rahmen des Projektstudiums an der Fachhochschule Trier im Fernstudiengang Informatik erarbeiteten Umsetzung eines Spiels weiterentwickelt wurde. **Suc26**

Sie erlaubt die Verwendung domänenspezifischer Handles, die zur Partitionierung von Systemgrenzen genutzt werden sollen. Durch die Einbindung zur Kompilierzeit erlaubt es die Engine so, statische Typinformationen zur Ableitung von Fragestellungen rund um Datenanomalien und Scheduling-Strategien zu nutzen. Gemeinsamkeiten und Abgrenzungen zwischen unserer Implementierung und dem Modell der Autoren auf.

2 Domänenspezifische Handles zur statischen Partitionierung und konfliktarmer Ausführung einer ECS-basierten C++-Game Engine

2.1 Forschungsfrage

Inwieweit können domänenspezifische Handles als statisch Partitionierungsgrenzen von Systemen dienen, dadurch unterschiedliche Laufzeitbereiche einteilen und eine laufzeitgünstige und konfliktarme Parallelisierung unterstützen?

2.2 Methodik

Die Forschungsfrage wird anhand eines Referenzszenarios beantwortet, das aus Kollisionssystemen und Partikelsystem mit einer hohen Objektanzahl besteht.

Die Systeme eignen sich für die empirische Betrachtung, da insbesondere Kollisionssysteme determinismuskritisch sind. Beide Systeme modellieren außerdem eine hohe Objektanzahl, was sich insbesondere für einen analytischen Vergleich in Hinsicht Skalierung eignet.

Hierbei betrachten wir die Bereiche Partitionierung, Systemkopplung und Scheduling-Konflikte und gehen dabei folgenden Unterfragen nach:

1. Architekturgrenzen Wir stellen fest, welche Architekturgrenzen sich durch die domänenspezifischen Handles über das Typsystem abbilden lassen.

2. Sichtbarkeit der Systemgrenzen im Scheduling Wir stellen fest, ob über die domänenspezifischen Handles Scheduling-Konflikte bereits statisch sichtbar oder vermeidbar werden.

3. Messung Dem empirischen Vergleich liegen folgende Messungen zugrunde: 1. A Unpartitionierte Welt ohne Parallelisierung 2. B unpartitionierte Welt mit Parallelisierung 3. C Partitionierte Welt mit Parallelisierung

Wir messen das Szenarium ohne Nebenläufigkeit, mit Nebenläufigkeit, und stellen dies einer domänenspezifischen Implementierung mit Scheduling gegenüber. Wir messen insgesamt 1. Laufzeitmessungen (FPS/Frametime) 2. Skalierung der Implementierungen bei wachsender Objektanzahl gemessen 3. Kompilierzeit 4. Code-Komplexität anhand statischer Code-Analyse-Tools. 5. Anzahl von Schedulingkonflikten

2.3 Vorläufige Gliederung

1. Einleitung und Zielsetzung 2. Grundlagen: ECS, datenorientierte Speicherlayouts und Nebenläufigkeit in Game Engines 3. *ECS Core* Modell und Abgrenzungen zu helios::ecs 4. Domänenspezifische Handles als statische Partitionierungsgrenzen 5. Das Relationale Modell als formales Fundament konfliktarmer ECS-Architekturen 6. Konfliktmodell für Systeme und Scheduling 7. Referenzszenario: Kollisions- und Partikelsystem 8. Implementierung der varianten 9. Evaluation: Laufzeit, Skalierung, Kompilierzeit, Code-Komplexität 10. Diskussion 11. Zusammenfassung und Fazit

3 Normalisierte ECS-Datenmodelle und domänenspezifische Handles zur statischen Konfliktanalyse von Systemen in einer C++-Game-Engine

3.1 Forschungsfrage

Wie kann normalisierte relationale Modellierung von ECS-Datenstrukturen genutzt werden, um Lese- und Schreibzugriffe von ECS-Systemen statisch zu beschreiben und mithilfe domänenspezifischer Handles Konflikte zwischen Systemen zu erkennen oder auszuschließen?

3.2 Methodik

Die Arbeit untersucht, ob domänenspezifische Handles im Zusammenhang mit einem relational modellierten ECS-Datenmodell dabei unterstützen, Scheduling-Konflikte auf Komponentenebene statisch zu analysieren.

Hierzu werden zunächst Gemeinsamkeiten und Abgrenzungen zwischen dem Query-Modell von Redmond et al. und der Implementierung in `helios::ecs` herausgearbeitet.

Der theoretische Teil motiviert anhand der Normalformtheorie im relationalen Modell eine entitätszentrierte

$$K(e, k_1, k_2, \dots, K_n)$$

sowie eine komponentenzentrische Zerlegung

$$K_1(e, k_1), \quad K_2(e, k_2), \quad \dots, \quad K_n(e, k_n)$$

von Entitäten und Komponenten. Auf dieser Grundlage wird die komponentenzentrische Speicherung als normalisierte Datenhaltung eingeordnet und die Verwendung von *Sparse Sets* als konkrete Speicherstruktur in `helios::ecs` begründet.

Im Anschluss werden ECS-Systeme über ihre Lese- und Schreibzugriffe auf Komponentenrelationen beschrieben. Dadurch entsteht ein statisches Zugriffsmodell, mit denen

Konflikte zwischen Systemen auf Komponentenebene identifiziert werden können.

Domänenspezifische Handles erweitern dieses Modell um typisierte System- und Laufzeitgrenzen. Hierüber soll untersucht werden, ob Konflikte bereits durch die Typisierung ausgeschlossen oder zumindest früher sichtbar gemacht werden können.

Ein weiterer Teil der Arbeit setzt diese Zugriffe in Beziehung zum formalen Scheduling-Modell von *ECS Core*, das zwischen serieller und nebenläufiger Ausführung unterscheidet. Hier soll diskutiert werden, ob sich für die betrachtete ECS-Implementierung Aussagen über konfliktarme beziehungsweise konfliktserialisierbare Systemausführung ableiten lassen.

3.3 Vorläufige Gliederung

1. Einleitung und Zielsetzung
2. Grundlagen: ECS, ERM, komponentenzentrische Speicherung und Systeme
3. Relationales ECS-Modell und Normalformanalyse
4. Abbildung auf `helios::ecs`: Komponenten, Stores und Queries
5. Systemzugriffe als Lese-/Schreibmengen
6. Domänenspezifische Handles als statische Systemgrenzen
7. Konfliktanalyse von ECS-Systemen
8. Referenzszenario: Kollisions- und Partikelsysteme
9. Evaluation: Statische Konfliktableitung im Referenzszenario
10. Diskussion
11. Fazit